

Secure Document Sharing System

By: Alina Nesimi and Mina Odabasi



Project Overview



- **The website/ssystem features:**

- **Authentication System**
 - Secure login and registration with password hashing
- **Encrypted File Storage**
 - Documents/files are encrypted before being stored
- **Document Sharing with Permissions**
 - Users can share files with permission like viewer or editor
- **Role-Based Access Control (RBAC)**
 - Different roles (admin, user, guest) enforce access restrictions
- **Audit & Monitoring**
 - Tracks all actions performed on documents
- **Security & Event Logging**
 - Security logs - Record suspicious activity and labels warning
 - Access Logs - Records normal user action throughout system



How it works:

1. User Registers
2. User logs in
3. Uploads file → file is then encrypted
4. File is stored on server
5. User can share file with others
 - a. Permissions determine access level
6. All actions are logged

Security Architecture

Layered Security Design:

- **Authentication:** Users must log in before accessing docs or uploading.
- **Authorization:** Role-based access (**admin, user, guest**).
- **Data Protection:** Files encrypted before storage.
- **Transport Security:** HTTPS (TLS/SSL)



Security Architecture – Authentication & Authorization

Authentication

Purpose: Verify user identity

Implementation example:

if not g.user:

return redirect(url_for("login"))

→ If not logged-in user automatically redirects to login page.

How it works:

- User logs in with username/password
- Session has a token stored in cookie
- Every request checks if user is authenticated

Why important: Authenticates who user is before logging someone in → not logged in = no access.

Authorization (Access Control)

Purpose: Control what users can do

Role-Based Access example:

@require_auth

@require_role("admin")

→ these are wrappers that check if user is logged in and has admin role

Role levels:

1. **Admin** → full access to system (view and manage all users and documents)
2. **User** → manage personal documents + access shared ones
3. **Guest** → limited access → can only read/view shared documents.

Permission Checks Example:

if not can_user_edit_document(doc, g.user):
abort(403)

→ function checks if you have permission based on role, and if you don't, then access won't be granted

Permission Levels:

- Owner → full control
- Editor → read and modify documents
- Viewer → read-only

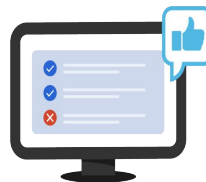
Why important: Ensures users only access what they are allowed to based on their

Authentication



Confirms users are who they say they are.

Authorization



Gives users permission to access a resource.

Security Architecture – Data Protection (Encryption)

Purpose: Protect any data stored on server like files and passwords

File Encryption:

```
file_bytes = file.read()
```

```
encrypted_data = cipher.encrypt(file_bytes)
```

→ Turns raw file data into encrypted format using cipher

Storing files:

```
with open(file_path, "wb") as f:
```

```
    f.write(encrypted_data)
```

→ saves these encrypted bytes to a file

Password Security:

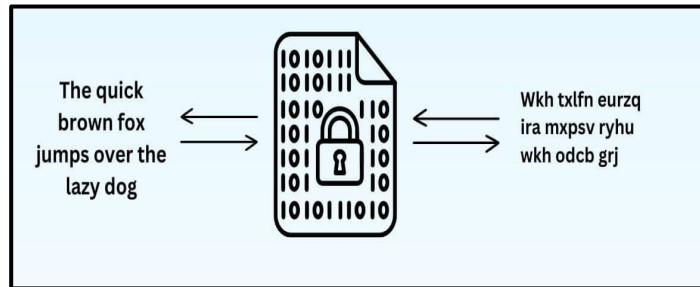
```
bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt())
```

 → Hashes

the password along with a salt so it cannot be reversed or uncovered easily.

Why important:

- Even if files are accessed directly → data is still unreadable
- Passwords are never stored in plain text



Security Architecture – Transport Security (HTTPS/TLS)

Purpose: Protect data in transit

Implementation: Flask runs with SSL certificate allowing HTTPS to be used instead of HTTP

Example: `app.run(ssl_context=("cert.pem", "key.pem"))`

- Runs server over HTTPS by enabling SSL/TLS encryption
- Uses a certificate and private key to ensure all data between the user and server is securely encrypted.
 - ◆ **Cert.pem** = public certificate which proves server's identity and is shared with users
 - ◆ **Key.pem** = private key that always stays secret on the server which is used to encrypt/decrypt data

Enforcement:

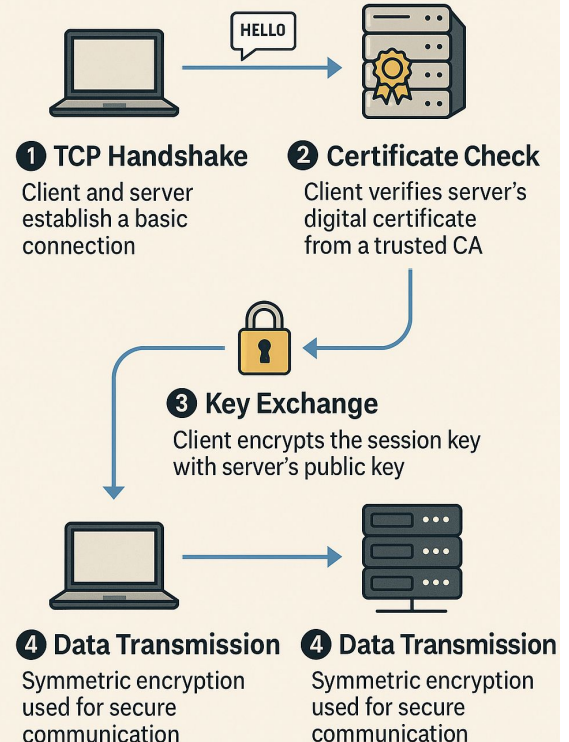
if not request.is_secure:
redirect to https

- Forces users to use secure connection (HTTPS if they try to access the site over HTTP)

Why important:

- Prevents interception → man in the middle attacks
- Encrypts credentials and data

How HTTPS Works



Key Security Implementations

Security implementations	Why there important?
Authentication (login system)	Ensures only verified/logged in users can access the system preventing unauthorized access
Authorization (role + permission checks)	Restricts what users can do based on roles (RBAC) in order to protect certain actions and data
File encryption (secure storage)	Converts files into ciphertext so that unreadable even if attackers gain direct access
Input/file validation (file & MIME checks)	Prevents malicious files from being uploaded
Password hashing	Protects user passwords by using (bcrypt → password hashing algo) which prevents password cracking
Logging (audit + security logs)	Tracks ALL system activity to be able to go back and investigate incidents
HTTPS (TLS/SSL encryption)	Encrypts data in transit between client and server and prevents man in middle attacks



Web Development Security Best Practices



Authentication

- Secure logins and strong passwords.



Authorization

- Restrict access by user roles.



Cryptography

- Encrypt sensitive data.



Input Validation

- Prevent SQL injection and XSS.



Logging & Monitoring

- Detect and respond to threats.



Challenges

1. MIME Type Validation

- Confusion between file content vs file extension importance.
- Understanding how to prevent spoofed files, files named .pdf but aren't actually pdf. (where MIME comes into play)
- Had issues with where to place `allowed_mime()` and how exactly to implement
- Learned that validation must happen **after getting the file but before saving**

2. Setting Up HTTPS / TLS

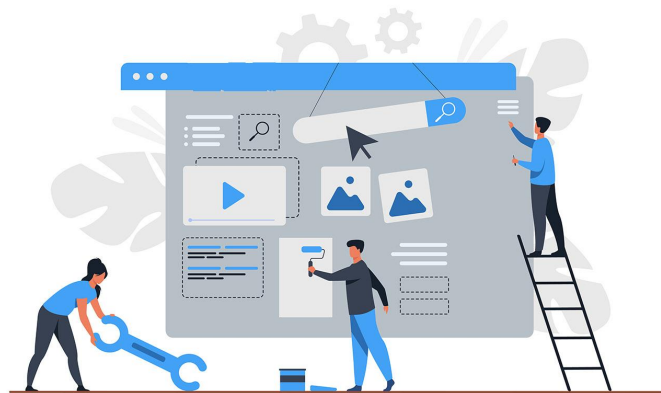
- Confusion between enabling HTTPS vs enforcing HTTPS
- Learned that `debug=False` does NOT enable HTTPS
 - only turns off Flask's debug mode
 - app can still run over plain HTTP unless HTTPS is separately configured.
- Trouble generating and understanding self signed certificates + browser warnings due to self-signed certificates

3. Handling Roles & Permissions

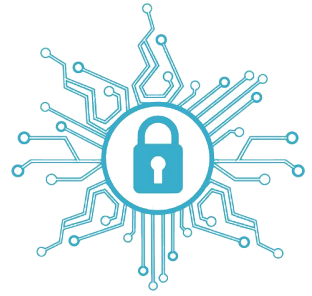
- It was a tedious process to assign each role different permissions and ensure every action had the correct access control.

4. Debugging 403 Errors

- “Forbidden” errors with unclear source
- Errors caused by incorrect permission checks and assignment of role right as well as mismatched routes .



Lessons learned:



1. Security must be layered
2. Wrappers allowed for reusability and makes code cleaner. Makes it easier to apply security checks multiple times.
3. `Url_for` → helps generate correct routes dynamically instead of hardcoding URL's
4. Small mistakes like misplaces routes and/or variable names can break functionality
5. Although I already knew backend controls security while frontend only reflects it this project taught how easily frontend restrictions can be bypassed and why strict backend checks are necessary for every request.